A Neural Microkernel: Interruptible, Verified Inference with a Self-Extending Deterministic Plane

Pierre Lamy
with Claude Code (Anthropic, Opus 4.8)

2026-06-22

Abstract

Large language models are deployed as uninterruptible black boxes: once a generation begins it runs to completion, its intermediate state is neither inspectable nor revertible, and any claim it makes about the world is taken on faith. We report an engineering study that wraps a real Mixture-of-Experts model (Cohere2-MoE / "North", run locally through MLX) in an operating-system-style *microkernel* whose organizing principle is a privilege boundary: the model *proposes*, a trusted deterministic plane *disposes*. The kernel contributes (i) a fail-closed scheduler with a three-gate admission contract and a capability/syscall model; (ii) a family of forward-pass mechanisms — a commit trap, KV-cache park/resume, expert-isolation, speculate/verify/rollback, and deterministic interruption — each proven to bit-exactness against real model activations; (iii) a *self-extending deterministic plane* in which the kernel detects a missing primitive, has the model synthesize it, and admits it only after a hold-out gate proves it correct on thousands of novel instances against an independent oracle (with a placebo control that rejects fail→pass-by-luck); (iv) an escalation solver that climbs greedy → temperature-diverse → repair → web-research → honest abstention, persisting every gate-proven helper; and (v) a small, hard-bounded, always-consulted persistent memory governed by a self-describing policy object. Validation comprises 19 CPU test suites (258+ assertions, all passing), six tensor-level invariant batteries that hold to bit-exactness on the live model, a 420-task coding benchmark run *through the kernel* (418/420; zero trace self-check violations), two adversarial multi-agent bug sweeps (21 and 4 reviewers; all confirmed findings fixed and regression-locked), and a demonstration that the same model solves two previously-failed tasks *generally* — hold-out-proven, with no memorization. We are explicit about what is and is not integrated: the forward-pass mechanisms are proven in isolation, while the running system today operates at whole-generation (scheduler) granularity; closing that gap is the principal future work.

1. Introduction

A modern language model is, operationally, a coroutine you cannot pause. It is invoked, it emits tokens until a stop condition, and it returns. While it runs there is no supervisor: no way to checkpoint the partial computation, no way to roll back a step that violated an invariant, no way to interpose a deterministic check between "the model decided x" and "x takes effect." Worse, the model's outputs are accepted on trust. When a model writes a helper function, or claims a fact, or reports that a test passed, nothing in the standard inference stack distinguishes a correct result from a confident hallucination.

Operating systems solved the analogous problem for untrusted user code decades ago. User processes do not get to decide whether their `write()` succeeds; they *request* it across a privilege boundary, and a trusted kernel — holding the only handle to the hardware — validates and disposes. The kernel does not need to *trust* the process because it *checks* the process. This paper asks what it takes to put that boundary around, and eventually inside, neural inference. The thesis, refined over the course of the work, is a single sentence: **the learned component proposes; a thin, general, trusted kernel disposes.** Critically — and this is the insight that shaped the architecture — *validation is a syscall*. Whether a generated primitive is correct, or a candidate fact is grounded, cannot be adjudicated by the same weights that produced it; it requires an independent, deterministic authority outside the model.

This report documents a working system built around that thesis and the experiments that validate it. Our contributions are:

1. **A fail-closed scheduler and capability model** (§2.2) with a three-gate admission contract: an operation executes only if it is registered, its capability has been granted, and a validator is installed — defaulting to refusal on any omission.
2. **Forward-pass kernel mechanisms** (§2.4) — commit trap, KV park/resume, switch-MLP expert isolation, speculate/verify/rollback, deterministic interrupt, and ring-buffer-crossing rollback — each validated to *bit-exactness* (residual ≡ 0) against the activations of a real 70B-class MoE.
3. **A self-extending deterministic plane** (§3) governed by a hold-out gate: the kernel grows new, *proven* capabilities at runtime without ever

trusting model output, because the trust verdict is computed in the parent against an independent oracle, never reported by the child.

4. **An escalation solver** (§3.4) that converts "the model failed once" into a disciplined ladder ending in either a hold-out-proven solution or an honest "I don't know," and that persists every helper it proves.
5. **A persistent, hard-bounded, always-consulted memory** (§4) carrying a self-describing governance object — a memory that tells the kernel how to use it.

We are deliberately conservative about claims. §5 separates what is proven at the tensor level from what is integrated into the running loop, and §6 states the limitations plainly.

2. Architecture

2.1 Three planes

The system is organized into three planes that mirror the proposer/disposer split:

- **The neural plane** — the model's forward pass. It is the *proposer*. Its outputs (next-token logits, expert routings, generated text) are treated as untrusted proposals.
- **The scheduler plane** — a deterministic task graph that admits, sequences, parks, resumes, and commits units of work. It is the *dispatcher*: it decides what runs and enforces the contracts under which it runs.
- **The deterministic plane** — pure, verifiable executors (parsers, transforms, validators, oracles, and the hold-out gate). It is the *disposer*: it is the only plane permitted to pronounce a result correct.

The privilege boundary runs between the neural plane and the other two. Anything the model produces must cross a gate to take effect.

2.2 The scheduler ABI and the three-gate contract

The scheduler exposes a deliberately small interface. Executors are registered as `register_executor(kind, fn)` where `fn(sched, task, inputs) -> (result, ok, ValidationLevel)`; validators as `register_validator(key, fn)` where `fn(result, task) -> bool`. Admission (`admit`) is **fail-closed** behind three independent gates:

1. **Operation exists** — the `kind` has a registered executor.
2. **Capability granted** — the task's declared capabilities are all present in the scheduler's grant set.
3. **Validator installed** — a validator is registered for the task's contract key, at a level no higher than the ceiling permitted for that operation.

A task that omits any of the three is rejected, not run. This is the structural reason the kernel can host side-effecting tools without being compromised by them: a tool with no granted capability, or no validator, simply cannot be admitted. Side-effecting tools additionally register a *syscall-op* descriptor (`_syscall_ops[name] = {capability, max_level}`), and admission enforces that the declared validator level does not exceed the tool's ceiling — so an I/O tool can never be silently promoted to a higher trust level than its capability warrants.

2.3 The trust spine: two tiers

The kernel recognizes exactly two tiers of trust, and never confuses them:

- **PROVEN (pure deterministic primitives).** A primitive earns this tier only by passing the hold-out gate (§3.1): correctness demonstrated on thousands of novel instances against an independent oracle. These are pure functions; they have no I/O and cannot lie about their effects.
- **CHECKED (syscall-tier tools).** Anything that touches the world — disk, network, the system clock, a subprocess — is capability-gated and its *result* is validated, but it is never marked PROVEN. The kernel cannot prove a network fetch "correct"; it can only check that the call was authorized and the result well-formed.

This distinction is enforced mechanically by the admission gates, not merely documented.

2.4 Forward-pass mechanisms

Five mechanisms give the kernel, in principle, OS-like control over a generation at the granularity of a single forward pass. Each is validated by an *invariant battery* — a test that asserts a residual is exactly zero on real model activations, not merely small:

- **Commit trap** — a checkpoint/abort primitive: a forward step can be speculatively taken and then either committed or reverted, with the post-revert state bit-identical to the pre-step state.

- **KV park/resume** — the key/value cache of a paused generation can be serialized ("parked") and later resumed such that continuation is bit-identical to never having paused.
- **Switch-MLP expert isolation** — in the MoE, masking the expert router so that only a chosen subset of experts fires produces activations identical to an independent reference computation over that subset (the masker ports the native `cohere2_moe` routing verbatim, rather than re-deriving it).
- **Speculate / verify / rollback** — a draft continuation can be proposed, verified against a condition, and rolled back on failure, leaving no residue in cache or RNG state.
- **Deterministic interrupt + ring-cross rollback** — a generation can be interrupted at an arbitrary step and rolled back deterministically, *including* the case where the interrupted span crosses a KV ring-buffer boundary (the >4096-token restore path), which is the hard case for cache restoration.

All six batteries (`inv_trap_plane`, `inv_kv_park`, `inv_switch_isolation`, `inv_speculate_rollback`, `inv_det_interrupt`, `inv_rollback_ringcross`) report residual ≡ 0 on the live North model.

Scope, stated honestly. These mechanisms are proven *in isolation*, at the tensor level. The integrated runtime today schedules at whole-generation granularity — it wraps complete generations rather than trapping every forward pass — and the in-model commit trap is exercised by its battery but not yet live-constructed inside the main loop. The GPU mega-kernel *fusion* of these mechanisms (sometimes called M5) is measured by a dynamics harness but not yet fused. Bridging from "mechanism proven" to "mechanism in the hot path" is the central item of future work (§6); we flag it here so no reader mistakes the invariant results for a fully-integrated forward-pass kernel.

3. The Self-Extending Deterministic Plane

The deterministic plane is not static. When the kernel encounters a task for which it has no proven primitive, it can *grow one* — and the discipline by which it does so without trusting the model is the heart of the contribution.

3.1 The hold-out gate

The hold-out gate is the kernel's notion of proof for a pure primitive. Given a candidate function, a *generator* of random inputs, and an independent *oracle*, the gate runs the candidate on `n` freshly generated instances (default in the thousands) and accepts only if every output matches the oracle. Three design choices make it trustworthy rather than merely a large test:

- **The trust boundary is asymmetric.** The generator and oracle come from the *gap specification* — the trusted source of truth. The model supplies *only the candidate*. The verdict — running the oracle and comparing — is computed in the trusted parent process. The untrusted child never grades itself; it returns raw outputs and nothing else. This defeats the obvious attacks (a candidate that patches `__eq__`, that re-implements the oracle to agree with itself, or that forges a "passed" report on stdout).
- **A novelty floor.** The gate counts *distinct* instances and refuses to certify if the generator did not actually produce enough variety, closing the loophole where a degenerate generator makes "1000 instances" mean one instance tested 1000 times.
- **A placebo control.** Because a single fail→pass transition can be luck or leakage, the gate re-runs under an independent seed and pairs each acceptance with a placebo check; *fail→pass alone is never accepted as proof*.

The gate is hardened against hostile candidates: leaf comparisons are type-gated (a candidate cannot smuggle an always-equal object through structural comparison), and the gate catches `BaseException` (so a candidate raising `SystemExit` cannot crash the gate into a false pass).

3.2 Sandboxed execution

Candidate code must run somewhere, and that somewhere is the most security-sensitive surface in the system. The venv tool launches each candidate in a subprocess under a macOS `sandbox-exec` profile that denies network and denies file writes outside a scratch directory, in its own process group (so a fork bomb is killed by `killpg`), under CPU/memory rlimits, with bytecode writing disabled. The launched child computes *raw outputs only*; the parent generates the instances, runs the oracle, and compares.

This division is what makes the sandbox sound: even a fully malicious candidate can at most return wrong numbers, which the gate then rejects — it cannot forge a verdict, because it never holds the verdict.

3.3 The extension pipeline

A `Gap` bundles everything needed to grow a primitive: an entry name, a natural-language description, sample cases, and the generator/oracle source. The `SelfExtender` drives the loop: synthesize a candidate, then `gate_candidate` — sample-test → hold-out gate → unsandboxed cross-check → second-seed re-gate → placebo — and, only on a clean sweep, register the primitive into the scheduler so it becomes callable *and* discoverable. Factoring `gate_candidate` into a single function ensures the pipeline and the escalation solver (§3.4) judge candidates *identically*.

3.4 The escalation solver

Real models fail on the first try. The escalation solver turns that failure into a ladder rather than a dead end. Each rung is judged by the same hold-out gate, so *no rung can win by memorizing the visible cases*:

Tier	Strategy	Rationale
1	Greedy, T = 0	One deterministic attempt.
2	Three diverse attempts at T = 0.8	Temperature-0 retries are byte-identical; real diversity requires sampling.
3	Repair each of the three, fed its own held-out failure	Targeted correction beats blind resampling.
4	Research: `web_search` + `fetch_page`, then build from the references	A knowledge gap may be closable with external material.
5	Abstain: *"I'm sorry, I don't know how to do that."*	An honest failure is preferable to a confident wrong answer.

Every gate-proven solution is persisted to a registry and registered for reuse. The research tier feeds *untrusted* web text into the model's prompt only — the resulting candidate must still clear the identical gate, so external content can influence *what* is proposed but never *whether* it is accepted.

3.5 Tools through the scheduler

All capabilities reach the model the same way: as scheduler-registered, capability-gated tools. The current catalog includes `datetime`; `sysinfo` (host memory/load/CPU/processor and free disk *for the running partition only* by default, with broader exposure behind an explicit flag, and unconditional secret redaction; plus scheduler state — process table, queue depths, held processes, memory slots, grants); `trace` (read live or persisted scheduler telemetry and run an invariant self-check); `web_search` / `fetch_page`; a delegate-and-verify tool (fire an agent to run a tool, then reconcile its result against the trace, with the verdict computed kernel-side rather than self-reported); and the memory tool of §4.

4. The Persistent Memory Store

The kernel is given a small, persistent memory with four properties the user specified: it is *always consulted*, *managed within hard bounds*, *kept up to date*, and *governed by a self-describing object that says so*. It is reached, like every other capability, through the scheduler as the capability-gated `op:memory`.

- **Persistent and durable.** SQLite in WAL mode, one file, surviving process restarts.
- **Hard-bounded (default 100 MB).** Every write is byte-accounted. If a write would exceed the cap, the store evicts least-recently-used *non-pinned* entries to make room; if it still does not fit — an oversized entry, or only pinned entries remain — the write is *rejected*. The cap is never exceeded, by construction.
- **A governance object.** A pinned, never-evicted entry holds the policy in plain language: *always consult this store before acting; record durable facts and proven helpers, update what changes, delete what becomes wrong; stay within the cap; store durable high-value items, not transient run state.* `consult()` always returns this object first — it is, literally, "the memory store object telling it to do so." The governance object cannot be deleted.

The tool dispatches `consult / get / put / list / delete / stats`. Because eviction is LRU over non-pinned entries, the policy itself and any explicitly pinned facts are protected from pressure.

5. Validation

We report five independent bodies of evidence. Where a result depends on the live model it is labelled as such; the rest is deterministic and reproducible on CPU.

5.1 Forward-pass invariants (live model)

The six invariant batteries of §2.4 all report residual ≡ **0** on the live North model: trap plane, KV park, switch-MLP isolation, speculate/rollback, deterministic interrupt, and ring-cross rollback (including the >4096-token restore path). Bit-exactness, not tolerance, is the bar — a single differing element fails the battery.

5.2 CPU test suites

Nineteen CPU suites pass in full (258+ assertions, zero failures):

Suite	Checks	Suite	Checks
task_graph	26/26	tools	19/19
lane_oracles	43/43	orchestrator	17/17
trace_tool	15/15	holdout_gate	12/12
self_extend	14/14	venv_tool	14/14
memory_store	14/14	investigation	13/13
agent_delegate	10/10	escalate_solve	10/10
kv_park	10/10	entry_trap	9/9
web_tools	8/8	commit_kernel	7/7
scheduler	4/4	det_interrupt	3/3
commit_trap	all		

These cover the scheduler ABI and admission gates, the hold-out gate's hostile-candidate hardening, the sandbox's verdict-in-parent design, the escalation ladder's tier order and abstention, the memory store's hard cap and eviction, and the fail-closed registration of every syscall-tier tool.

5.3 Coding benchmark through the kernel

To confirm that the kernel additions preserved end-to-end coding ability, we ran the full coding benchmark **through the kernel, on the live model via MLX** (not through the auxiliary JavaScript harness): 420 tasks

across five suites. Result: **418/420**, with **zero trace self-check violations** reported by the analyzer on any suite.

Suite	Pass	Suite	Pass
warmups	49/50	ladder	100/100
challenge	199/200	expert	50/50
property	20/20		

The two misses were a single greedy draft on terse prompts; §5.4 shows the same model solves them generally once escalation and the gate are applied. The benchmark path is structurally independent of the new kernel modules, so a regression there was not merely absent but not possible by construction — the new code is not on that path.

5.4 Solving held-out challenges without memorization (live model)

The two benchmark misses — `to_snake_case` (w47) and a Redis-RESP response parser (c156) — were re-attempted under the escalation solver, each judged by the hold-out gate at 2000 novel instances per seed across two seeds, with a placebo control. Both were solved *generally*:

- **w47** solved at **tier 1** (greedy); 2000/2000 on two independent seeds.
- **c156** solved at **tier 2** — tier 1 failed the sample, and a temperature-0.8 diverse attempt passed 2000/2000 on two seeds. The root cause of the original failure was identified (a top-level result did not strip a trailing CR-LF that a nested helper did handle).

Both were persisted. The research tier was independently exercised end-to-end: a live `web_search` + `fetch_page` retrieved ~5 KB of the genuine RESP specification, which the model consumed before the gate ran — confirming the kernel can *research and build*, not merely *recall*. Because the generator and oracle are independent of the candidate and the instances are novel, passing the gate is evidence of a general algorithm, not of memorized cases.

5.5 Adversarial bug sweeps

The new surface was audited by two multi-agent sweeps in which independent reviewers proposed findings and a second wave attempted to *refute* each one (defaulting to “refuted” when unsubstantiated):

- **Core plane sweep (21 reviewers).** Fourteen findings verified; thirteen actionable issues fixed and regression-locked — among them the hold-out gate's hostile-`__eq__` and `BaseException` paths, an admission-ceiling that could be skipped via a null validator, the venv tool's stdout-forge and unclamped resource limits, and the delegate tool's self-graded verdict.
- **Escalation surface sweep (4 reviewers).** All four modules returned *ship*; zero CRITICAL/HIGH survived verification. The central security claim was checked and held — *no unproven solution can be registered or persisted*, because `solved = True` is reachable only through the gate. Two MEDIUM issues were fixed: a result-sentinel that could collide with attacker-controlled page text (anchored to a newline the genuine emit guarantees, which `JSON.stringify` cannot produce), and a registry loader that did not type-check its input. A server-side-request-forgery exposure in the underlying web tool (private/metadata addresses, redirects) was identified and tracked as out-of-scope for the bridge.

5.6 Grounding and calibration (live model, related result)

A related experiment gave a smaller model (Mellum) kernel-owned, read-only fixture investigation (grep/read, recon-seeded) on a 50-task cyber benchmark, lifting raw pass rate 22→25/50. The more important effect was *calibration*: nine of the remaining misses became honest "insufficient evidence" abstentions rather than confident wrong answers. We take this as corroboration of the paper's stance that a kernel's value is as much in disciplining *when the model declines* as in raising the raw score.

6. Discussion and Limitations

What the kernel buys. The recurring pattern is *learned-proposes / kernel-disposes* applied at three granularities: token/expert selection (the forward-pass mechanisms), primitive synthesis (the hold-out gate), and task solving (the escalation ladder). In every case the model's creativity is retained and its authority is removed; correctness is adjudicated by a deterministic authority the model cannot influence. The methodological companion to this is *adversarial verification*: every claimed bug, and every claimed proof, is checked by an independent process biased toward refutation.

What is not yet integrated. The honest gap is between the forward-pass mechanisms (proven in isolation, §5.1) and the running loop (which schedules whole generations). A live, per-forward-pass commit trap and the fused mega-kernel are measured but not yet in the hot path. Until that bridge is built, the strongest claims — interruption and rollback *during* a live generation under load — are supported at the mechanism level, not the system level.

Other limitations. The web tool's SSRF surface is real and tracked. The persistent memory is single-process (SQLite WAL); concurrent multi-process writers are out of scope. The escalation solver's research tier helps only when the failure is a *knowledge* gap; for a pure coding bug (as in c156) it executes but adds little over temperature diversity. And two of 420 benchmark tasks still require escalation rather than a single draft — expected for one-shot greedy decoding on terse prompts, but a residue nonetheless.

7. Conclusion

We have described a neural microkernel that treats a real MoE model as an untrusted proposer behind a trusted, deterministic disposer. The kernel admits work through a fail-closed three-gate contract, grows new *proven* capabilities at runtime via a hold-out gate that computes its verdict outside the model, escalates failures through a disciplined ladder to either a proven solution or an honest abstention, and remembers within hard, self-governed bounds. The evidence — bit-exact forward-pass invariants, 19 fully-passing CPU suites, 418/420 on a kernel-routed coding benchmark with zero trace violations, two adversarial sweeps with all confirmed findings fixed, and general, hold-out-proven solutions to previously-failed tasks — supports the central claim at the level of mechanisms and scheduler-plane integration. The remaining work is to carry the proven forward-pass mechanisms into the live generation loop, at which point "interruptible, verified inference" becomes a property of the running system and not only of its parts.

Appendix A. Reproduction

CPU validation is deterministic and self-contained:

```
cd finetune/gpt-oss-unsloth/scripts
PYTHONPATH="$PWD" python kernel/test_*.py        # 19 suites
```

The live-model results require the North weights and MLX; the coding benchmark is driven by `kernel/run_coding_kernel.py --model north`, and the escalation demonstrations by `kernel/escalate_solve.py --task both --n 2000`. Telemetry is written under `reports/telemetry/` and audited with `kernel/analyze_telemetry.py`.

Appendix B. Module map

Concern	Module
Scheduler ABI, admission gates	`kernel/task_graph.py`
Forward-pass commit trap	`kernel/commit_trap.py`
Hold-out gate	`kernel/holdout_gate.py`
Sandboxed candidate execution	`kernel/venv_tool.py`
Self-extension pipeline	`kernel/self_extend.py`
Escalation solver	`kernel/escalate_solve.py`
Web-tool bridge	`kernel/web_tools.py`
Delegate-and-verify	`kernel/agent_delegate.py`
Syscall-tier tool registry	`kernel/tools.py`, `kernel/tools_builtin.py`
Trace / self-check	`kernel/trace_tool.py`, `kernel/analyze_telemetry.py`
Persistent memory	`kernel/memory_store.py`
Model adapter (MLX)	`north_adapter.py`